Agile Development and Extreme Programming

Svetlin Nakov

National Academy for Software Development

academy.devbg.org

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

Agenda

- Development Methodologies
- Agile Development
- Extreme Programming (XP)
  - How It Works for Me?
- Develop Your Own Methodology

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

Development Methodologies

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

What is a Methodology?

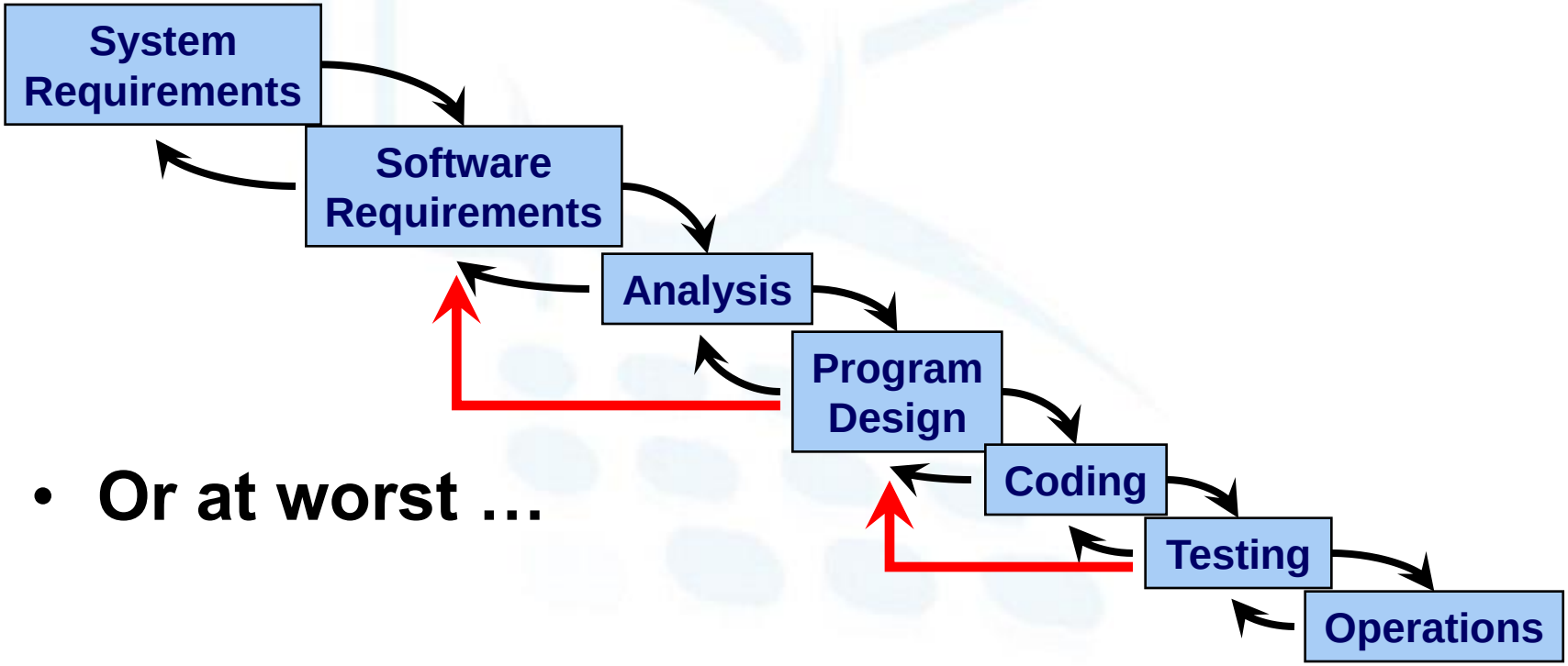
- A ***methodology*** is a formalized process or set of practices for creating software
  - A set of rules you have to follow
  - A set of conventions the organization decides to follow
  - A systematical, engineering approach for organizing software projects

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

The Waterfall Development Process

The Waterfall Process

- **The traditional development process:**

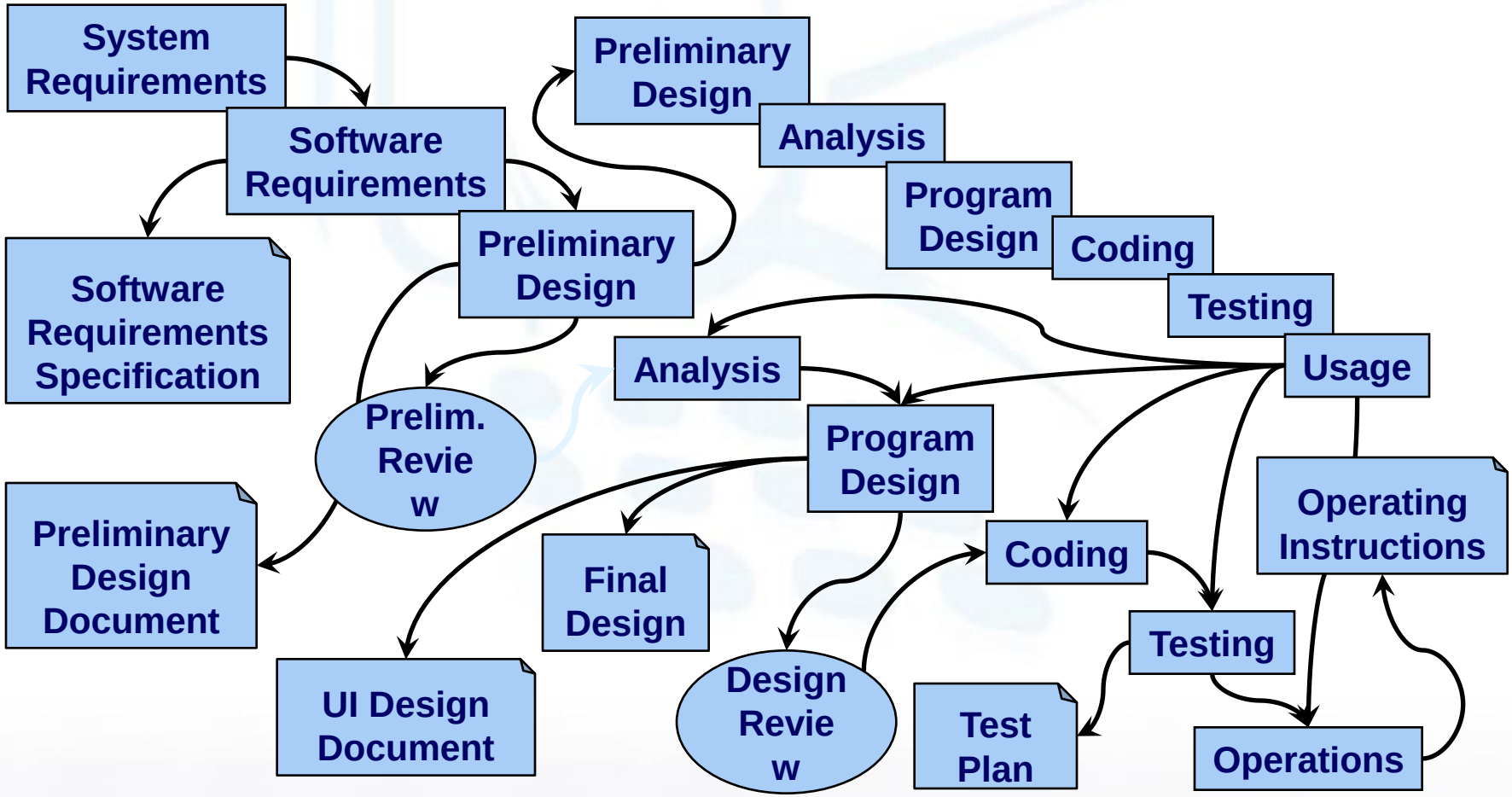


- **Or at worst …**

- **But this always ends up happening!**

Formal Processes

- **Formal efforts to “fix” the problem**



NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

Agile Development

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

Agile Manifesto

“Our highest priority is to satisfy the customer through early and continuous delivery of valuable software“

[Manifesto for Agile]

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

The Agile Spirit

- Incremental
  - ***Working software*** over comprehensive documentation
- Cooperation
  - ***Customer collaboration*** over contract negotiation
- Straightforward
  - ***Individuals and interactions*** over processes and tools
- Adaptive
  - ***Responding to change*** over following a plan

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

Agile Methodologies

- eXtreme Programming (XP)
- Scrum
- Crystal family of methodologies
- Feature-Driven Development (FDD)
- Adaptive Software Development (ASD)
- Dynamic System Development Model (DSDM)
- Agile Unified Process (AUP)

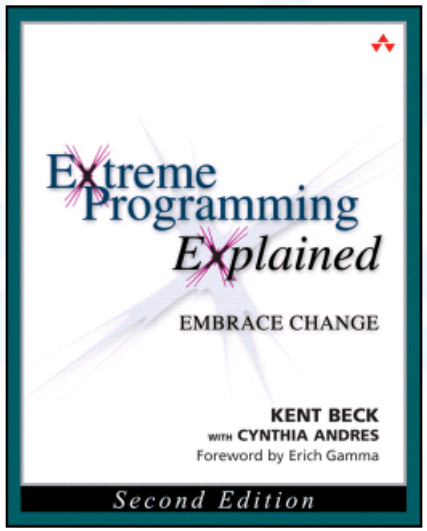
NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

eXtreme Programming

The XP Guru: Kent Beck

- **eXtreme Programming**
  - **The most prominent agile development methodology**

Kent Beck

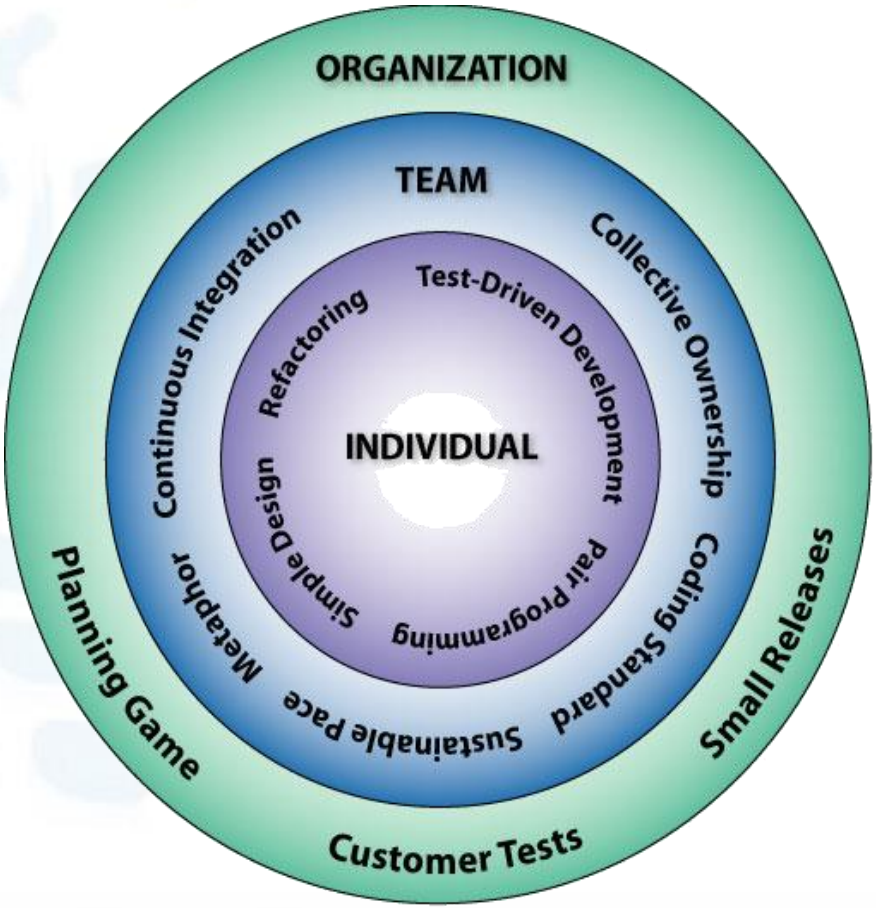


1st ed. Oct 1999

2nd ed. Nov 2004

The 12 Key Practices

- The Planning Game
- Small Releases
- Metaphor
- Simple Design
- Test-Driven Development
- Refactoring
- Pair Programming
- Collective Ownership
- Continuous Integration
- 40-Hour Workweek
- On-site Customer
- Coding Standards



NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

1. Metaphor

- **Guide all development and conversations with a simple shared story of how the whole system works**
  - **Gives the team a whole picture of describing the system, where new parts fit, etc.**
- **Words used to identify technical entities should be chosen from the metaphor**
- **The default metaphor is the business domain, and it’s usually just fine**

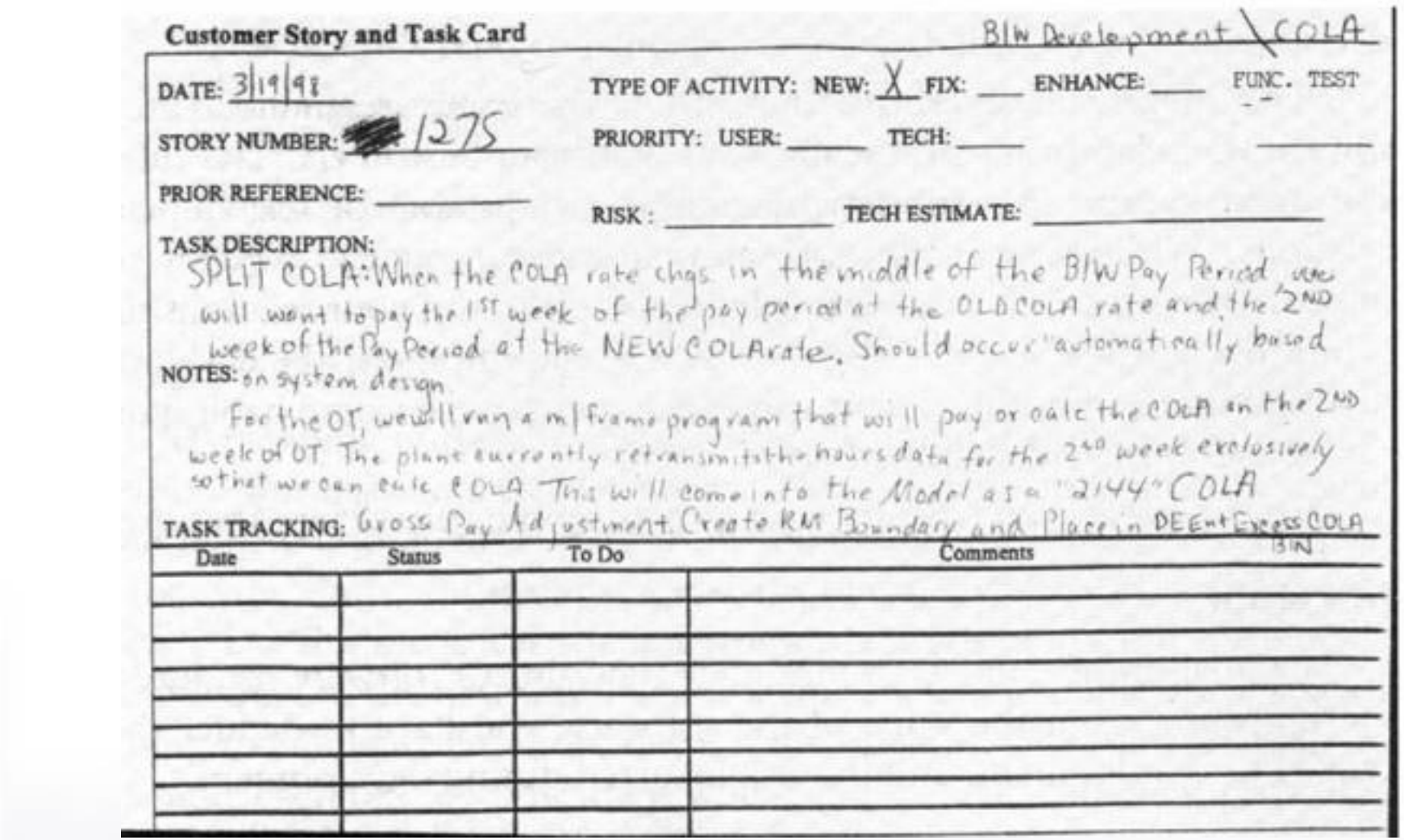
How It Works for Me?

- **Metaphors are good idea**
- **People should know the business needs and how their work fits in the project**

2. Release Planning

- Requirements via *User Stories*
  - Short cards with natural language description of what a customer wants
  - Prioritized by customer
- Resource and risk estimated by developers
  - Via “The Planning Game”
- Play the Planning Game after each increment

User Stories



Customer Story and Task Card — B/W Development / COLA

DATE: 3/19/98 — TYPE OF ACTIVITY: NEW: X FIX: ___ ENHANCE: ___ FUNC. TEST ___

STORY NUMBER: [redacted] 1275 — PRIORITY: USER: ___ TECH: ___

PRIOR REFERENCE: ___ — RISK: ___ TECH ESTIMATE: ___

TASK DESCRIPTION:
SPLIT COLA: When the COLA rate chgs in the middle of the B/W Pay Period we will want to pay the 1ST week of the pay period at the OLD COLA rate and the 2ND week of the Pay Period at the NEW COLA rate. Should occur automatically based on system design

NOTES:
For the OT, we will run a m/frame program that will pay or calc the COLA on the 2ND week of OT. The plane currently retransmits the hours data for the 2ND week exclusively so that we can calc COLA. This will come into the Model as a "2144" COLA BIN.

TASK TRACKING: Gross Pay Adjustment. Create RM Boundary and Place in DEEntExpressCOLA

Date	Status	To Do	Comments

How It Works for Me?

- Requirements specification (SRS) is better than user stories
  - Written documentation works well for large projects
- I prefer prototyping the user interface as source of documentation
- Sometimes its is hard to estimate the required resources
- Small releases have less risk

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

3. Testing

- Test-Driven Development (TDD)
  - Write tests before code
  - Tests are automated
  - Often use xUnit framework
  - Must run at 100% before proceeding
- Acceptance Tests
  - Written with the customer
  - Acts as “contract”
  - Measure of progress

Test-Driven Development

- Developers write unit tests before coding
- Motivates coding
  - Improves design: cohesion and coupling
  - Provides regression tests
  - Provides specification by example

```
public void TestMultiplication() {
  Dollar five = Money.dollar(5);
  AssertEqual(new Dollar(10), five.times(2));
  AssertEqual(new Dollar(15), five.times(3));
}
```

How It Works for Me?

- TDD is good for most projects, not for all
  - The real world is different: you always need the functionality "for tomorrow"!
- I use unit testing for complex logic only
  - Testing simple logic is overhead

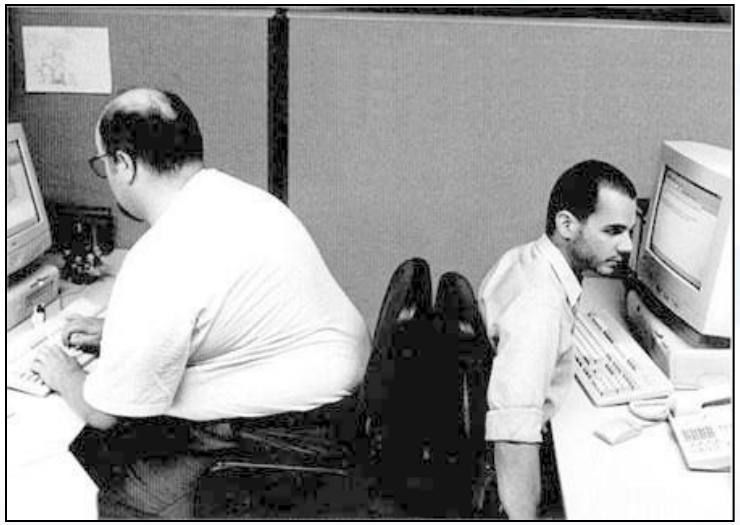
4. Pair Programming

- **Two software engineers work on one task at one computer**
  - ***The driver*** **has control of the keyboard and mouse and creates the implementation**
  - ***The navigator*** **watches the driver's implementation**
    - **Identifies defects and participates in on-demand brainstorming**
  - **The roles of driver and observer are periodically rotated**

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

Pair Programming

- Pairs produce higher quality code
- Pairs complete their tasks faster
- Pairs enjoy their work more
- Pairs feel more confident in their work



NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

How It Works for Me?

- Pair programming is great for complex and critical logic
  - When developers need good concentration
  - Where quality is really important
  - Especially during design
  - Reduces time wasting
- Trivial tasks can be done alone
- Peer reviews instead pair programming is often alternative

5. Refactoring

- Improve the design of existing code without changing its functionality
  - Relies on unit testing to ensure the code is not broken
- Bad smells in code:
  - Long method / class
  - Duplicate code
  - Methods does several different things (bad cohesion)
  - Too much dependencies (bad coupling)
  - Complex / unreadable code

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

How It Works for Me?

- Delivering working software faster is important!
  - You can write the code to run somehow
    - With simple design
    - With less effort
  - Later you can refactor the code if necessary
- Refactoring is not a reason to intentionally write bad code!
  - Good coding style is always important!

6. Simple Design

- No Big Design Up Front (BDUF)
  - Reduces the overhead
  - Ship working functionality faster and get feedback early
- “Do The Simplest Thing That Could Possibly Work”
  - Later use refactoring to change it
- Not too much formal documentation

How It Works for Me?

- Simple design does not mean "no design"
  - It is about establishing priorities
  - It's a set of tradeoffs you make
  - If something is important for this release and for the whole system, it should be designed well
  - Don't lose time to design something you will not use soon!

7. Collective Code Ownership

- Code belongs to the project, not to an individual engineer!
- Any engineer can modify any code
- Better quality of the code
- Engineers are not required to work around deficiencies in code they do not own
  - Faster progress
  - No need to wait for someone else to fix something

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

Collective Code Ownership?

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

How It Works for Me?

- **Collective code ownership is absolutely indispensable**
  - **You need to fight the people who don't agree with this!**
  - **Fire people writing unreadable and unmaintainable code**
  - **Don't allow somebody to own some module and be irreplaceable**

8. Continuous Integration

- Pair writes up unit test cases and code for a task (part of a user story)
- Pair unit tests code to 100%
- Pair integrates
- Pair runs ALL unit test cases to 100%
- Pair moves on to next task with clean slate and clear mind
- Should happen once or twice a day

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

How It Works for Me?

- **Integrating often is really valuable**
  - **Sometimes you cannot finish a task for one day and integrate it**
  - **For small projects with small teams integration is not an issue**
  - **For large and complex projects it's crucial**

9. On-Site Customer

- Customer available on site
  - Clarify user stories
  - Make critical business decisions
- Developers don’t make assumptions
- Developers don’t have to wait for decisions
- Face to face communication minimizes the chances of misunderstanding

How It Works for Me?

- On-site customer actually does not work!
  - Customers are busy
    - Meetings every day is working better
  - Customers are not competent!
    - Customers always say "Yes, this is what I want" and later say the opposite
    - You need to think instead of them
    - Use prototyping

10. Small Releases

- Timeboxed
- As small as possible, but still delivering business value
  - No releases to ‘implement the database’
- Get customer feedback early and often
- Do the planning game after each iteration
  - Do they want something different?
  - Have their priorities changed?

How It Works for Me?

- Small releases are really valuable
  - Manage the risk of delivering something wrong
  - Helps the customer to define better requirements
- Release every few weeks
- Large projects are not so flexible
  - Try to release something, even you know that it will be changed

11. Forty-Hour Work Week

- Kent Beck says, “ . . . fresh and eager every morning, and tired and satisfied every night”
- Burning the midnight oil kills performance
- Tired developers make more mistakes
  - Slows you down more in the long run
- If you mess with people’s personal lives (by taking it over), in the long run the project will pay the consequences

How It Works for Me?

- 40 hours a week or 40 hours without a sleep?
  - Come back to the real world!
  - Overtime is not recommendable but often can not be avoided
- Better planning can help
- Highly skilled senior engineers always suffer of overtime and high pressure
  - That's how the business works!

12. Coding Standards

- Use coding conventions
  - Rules for naming, formatting, etc.
  - Write readable and maintainable code
- Method commenting
  - Self-documenting code
  - Don't comment bad code, rewrite it!
- Refactor to improve the design
- Use code audit tools (FxCop, CheckStyle, TFS)

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

How It Works for Me?

- **Coding standards are important**
  - **Enforce good practices to whole the team – tools, code reviews, etc.**
- **Standards should be simple**
  - **Complex standards are not followed**
  - **Standards should be more strict for larger teams**
  - **Developers don't like utter rules like "*comment any class member*"**

The 13th Practice? The Stand Up Meeting

- Start the day with 15-minute meeting
  - Everyone stands up (so the meeting stays short) in circle
  - Going around the room everyone says specifically:
    - What they did the day before
    - What they plan to do today
    - Any obstacles they are experiencing
  - Can be the way pairs are formed

People Communicate Most Effectively Face-to-Face

Communication effectiveness

Paper

2 people on email

Videotape

2 people on phone

2 people at whiteboard

Richness of the communication channel

How XP Solve Some SE Problems

Problem	Solution
Slipped schedule	Short development cycles
Cancelled project	Intensive customer presence
Cost of changes	Extensive, ongoing testing, system always running
Defect rates	Unit tests, customer tests
Misunderstand the business	Customer part of the team
Business changes	Changes are welcome
Staff turnover	Intensive teamwork

So What does XP Apply to?

- Domains with changing requirements
- High-risk projects (including those with high schedule risk)
- Small project team: 2 – 12 programmers
  - Cannot be used with a large team
- Extended development team
  - Developers, managers and customer
  - Co-located
- Automated testability

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

Your Own Development Process?

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

Mix-and-Match

- The practices in different agile methods can be extracted and combined
- Establish your own process
  - Build it step-by-step
  - Adapt good practices one by one
- Examples
  - Pair programming and its variation
  - Daily 15-minutes meeting
  - Test-driven development

NASD
NATIONAL ACADEMY FOR SOFTWARE DEVELOPMENT

Agile Development and XP